# Delivery 2: Governance & Technical Debt Audit

## Governance Pipeline:

*Configure a CI workflow (GitHub Actions) that measures **Cyclomatic Complexity** and **Code Coverage**. It must fail/warn if metric degrades.*

### 1. Crear una rama

En mi caso lo llame `siqui`

### 2. Crear archive workflow

Este directorio es indispensable para nuestro workflow.

`.github\workflows\`**`test_and_build.yaml`**

### 3. Get started with Github Actions

Buscamos Python applions en Github actions como nuestro workflow automatizado. Ejecutará nuestras tareas de build,test y deploy de nuestro código

Copiamos y pegamos el workflow code básico a al archivo creado anteriormente.

### 4. Instalar Coverage

Nuestro proyecto necesita generar coverage (normalmente Jest ya lo hace). Pero en nuestro caso comono existía coverage, instalamos;

```
npm install --save-dev jest
```

## 5. Modificar jobs

Primero quisimos cambiar el nombre del build y luego la versión de Python a la que tenemos ya previamente instalado.

```yaml
# What the worflow must execute when triggered
jobs:
  #Renamed build-->"text-and-build"
  test-and-build:

    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4
    - name: Set up Python 3.12.7
      uses: actions/setup-python@v3
      with:
        python-version: "3.12.7"
```

Actualiza pip, Instalar herramientas de calidad (flake8), Instala framework de tests (pytest) e Instala dependencias del proyecto.

```yaml
- name: Install dependencies
    run: |
      python -m pip install --upgrade pip
      pip install flake8 pytest
      if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
  - name: Lint with flake8
```

Ejecuta la suite de pruebas del sistema con pytest, Valida que el sistema sigue funcionando después de cambios.

```
- name: Lint with flake8
```

Verifica el umbral de cobertura: si la cobertura es inferior al 70 %, la compilación fallará.

```
- name: Check coverage threshold
  run: |
      COVERAGE=$(node -p "require('./coverage/coverage-summary.json').total.lines.pct")
      echo "Coverage is $COVERAGE%"
      if (( $(echo "$COVERAGE < 70" | bc -l) )); then
        echo "Coverage below threshold"
        exit 1
      fi
```

## 6. Instalar ESLint

ESLint es una herramienta de código abierto enfocada en el proceso de "lintig" para javascript (o más correctamente para ECMAScript). ESLint es la herramienta predominante para la tarea de "limpiar" código javascript tanto en el servidor (node.js) como en el navegador.

```
npm install --save-dev eslint
```

Crear nuevo archivo llamado `.eslintrc.json` para agregarle luego este código

```json
{
  "extends": "eslint:recommended",
  "rules": {
    "complexity": ["warn", 10]
  }
}
```

Agregar en `package.json` el script.

```
"lint": "eslint ."
```

Agregar este nuevo job al haber configurado lo anterior.

```yaml
# Run the complexity analysis with eslint
- name: Run complexity analysis
  run: npm run lint

```

**Esto lanza warning si la complejidad > 10.**

Posterior, se muestra la configuración para SonarQube cloud, donde se analizan las Qualiti gates en nuestro proyecto.

Se debe registrar `SONAR_TOKEN` en `Secrets` Variables en Github y luego proveer los argumentos que son el nombre de la organización y el `proyectKey` proveídos y configurados en SonarQube cloud.

```yaml
# Run SonarQube Scanner
  - name: SonarCloud Scan
    uses: SonarSource/sonarqube-scan-action@v6
    env:
      SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
    with:
      args: >
        -Dsonar.organization=conduit-realworld-django
        -Dsonar.projectKey=Pablokill2004_Ing.-de-Software---Proyecto
```